

Proiect - Învățare automată

Rotaru Claudiu-Cristian (TIA I)

Conf. dr. ing. Dan Munteanu

Capitolul 1. Clasificare	3
1.1.Definiție	3
1.1.1. Proces.....	3
1.1.2. Algoritmi.....	3
1.1.3. Metrici de evaluare	4
1.2.Aplicație	4
1.2.1. Atribute	4
1.2.2. Prelucrarea setului de date	5
1.2.3. Alegerea si compararea modelelor.....	7
1.2.4. Fine tuning	8
1.2.5. Rezultate	10
1.2.6. Compararea performantei	11
1.2.7. Concluzii	12
Capitolul 2. Regresie	12
2.1.Definiție	12
2.1.1. Proces.....	12
2.1.2. Algoritmi.....	12
2.1.3. Metrici de evaluare	13
2.2.Aplicație	13
2.2.1. Atribute	13
2.2.2. Prelucrarea setului de date	13
2.2.3. Modele alese	14
2.2.4. Fine tuning	14
2.2.5. Rezultate	16
2.2.6. Performanta algoritmilor	18
2.2.7. Concluzii	18
Capitolul 3. Clusterizare	18
3.1.Definiție	18
3.1.1. Proces.....	18
3.1.2. Algoritmi.....	19
3.2.Aplicație	19
3.2.1. Atribute	19
3.2.2. Prelucrarea setului de date	20
3.2.3. Modele alese	20
3.2.4. Fine tuning	20
3.2.5. Rezultate	21
3.2.6. Concluzii	23
Bibliografie	24

Capitolul 1. Clasificare

1.1. Definiție

Clasificarea în învățarea automată (Machine Learning) este o metoda fundamentală de învățare supervizată prin care un algoritm analizează date istorice gata etichetate pentru a învăța cum să atribuie o categorie unor date noi. Modelul învață tiparele din trecut pentru a face predicții corecte în viitor.

1.1.1. Proces

Procesul de clasificare consta în următorii pași:

- colectarea și prelucrarea datelor: se folosesc caracteristici input și caracteristici țintă
- antrenarea modelului: datele se separă în date de antrenament și de testare. Datele de antrenament sunt folosite pentru a găsi tiparele ce separă clasele între ele
- testarea și validarea: datele de testare sunt cele care nu au fost văzute în timpul antrenării și sunt date modelului pentru clasificare. Performanța este măsurată comparând predicțiile cu valorile reale.

1.1.2. Algoritmi

Cei mai populari algoritmi de clasificare sunt:

- regresia logistica
- arborii de decizie
- random forest
- support vector machine
- k-nearest neighbours

1.1.3. Metrice de evaluare

Evaluarea algoritmului se face prin mai multe metrice incluzând:

- **Acuratețe** – Proporția de predicții corecte din total
- **Precizia (Precision)** – Proporția de elemente clasificate drept pozitive care sunt cu adevărat pozitive (previne alarme false)
- **Sensibilitatea (Recall)** – Cate dintre elementele pozitive au fost identificate corect de model (previne ratarea cazurilor importante)
- **F1-Score** – Media armonica între precizie si recall. Oferă o imagine de ansamblu
- **Matricea de confuzie** – Tabel ce arata clar numărul de predicții corecte (True Positive si True Negative) si greșite (False Positives, False Negatives).

1.2. Aplicație

Pentru secțiunea de clasificare a proiectului am ales setul de date heart.csv, ce conține un număr de înregistrări ce conțin diverși factori, măsurători ale funcțiilor corpului si simptome

Sursa: <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction/data>

1.2.1. Atribute

Atributele sunt:

- **Age:** Vârsta pacientului
- **Sex:** Sexul pacientului
- **ChestPainType:** Tipul durerii în piept
- **RestingBP:** Tensiunea arterială în stare de repaus
- **Cholesterol:** Nivelul de colesterol seric
- **FastingBS:** Glicemia pe nemâncate
- **RestingECG:** Rezultatul electrocardiogramei în repaus
- **MaxHR:** Frecvența cardiacă maximă atinsă
- **ExerciseAngina:** Angină indusă de efortul fizic
- **Oldpeak:** Valoarea numerică a depresiei segmentului ST indusă de efort comparativ cu starea de repaus.
- **ST_Slope:** Panta segmentului ST la efort maxim
- **HeartDisease:** Variabila țintă / Diagnosticul (**1**: prezența bolii cardiace, **0**: sănătos).

1.2.2. Prelucrarea setului de date

În primul rând, setul de date este curățat de valorile fizic imposibile (colesterol 0, tensiune 0)

Am luat în considerare limitarea valorilor în intervale posibile însă nu am continuat cu aceasta din diverse motive:

Pentru colesterol există un interval destul de larg de valori (între 50-1500)

Pentru bătăi pe minut de 60 după testul de alergare pe bandă, nu putem ști dacă pacientul are medicamente ce încetinesc bătăile, dacă nodul sinoatrial este problematic, dacă echipamentul a măsurat doar jumătate dintre bătăi sau dacă personalul medical a înregistrat 60 în loc de 160

În al doilea rând am convertit stringuri în valori binare fie direct (m-1, f-0) fie prin crearea de noi coloane (o coloană cu 3 valori posibile a fost transformată în 3 coloane tip boolean pentru fiecare valoare posibilă)

Am separat coloanele input de coloanele target (HeartDisease vs restul)

Am separat datele de antrenare și cele de testare în raport de 80:20

Am standardizat valorile continue în intervale centrate în jurul valorii medii 0 și cu deviația standard 1 pentru a egala importanța fiecărei variabile

Primul test constă în compararea acurateții în cazul datelor nefiltrate (colesterol = 0, tensiunea arterială = 0) vs eliminarea acestor rânduri vs schimbarea valorilor imposibile cu media setului.

În cazul nefiltrat, Decision Tree este câștigătorul într-o cursă strânsă (~1% diferență)

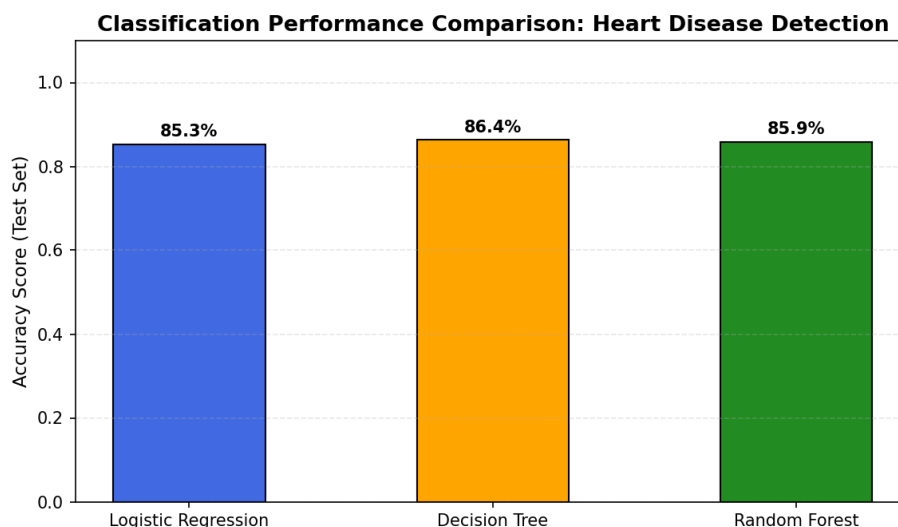


Fig. 1 Acuratete date nefiltrate

In cazul filtrat, Decision Tree-ul suferă cel mai mult din cauza numărului redus de date (20% mai puține), pe când regresia logistica si random forest ating acuratețe de 90, respectiv 90.7%

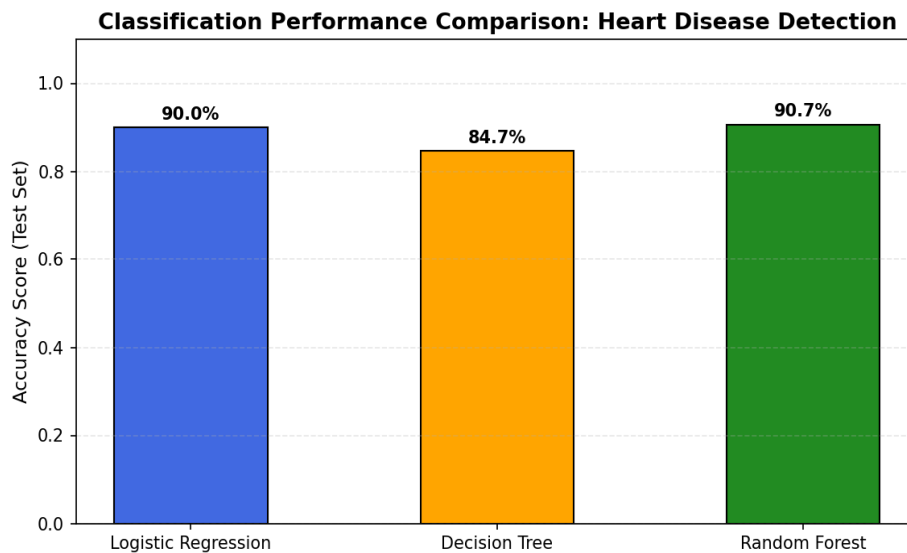


Fig. 2 Acuratete date filtrate

In cazul valorilor schimbate, am înlocuit valorile nule cu media, folosind SimpleImputer din sklearn.impute. Asta ne menține numărul de rânduri, însă are de suferit acuratețea originală a datelor. Rezultatele arata ca arborele de decizie suferă mai mult, printr-o scădere de 4%, regresia câștigă 1.1% iar random forest câștigă 2.1%, măsurate fata de baseline (nefiltrate).

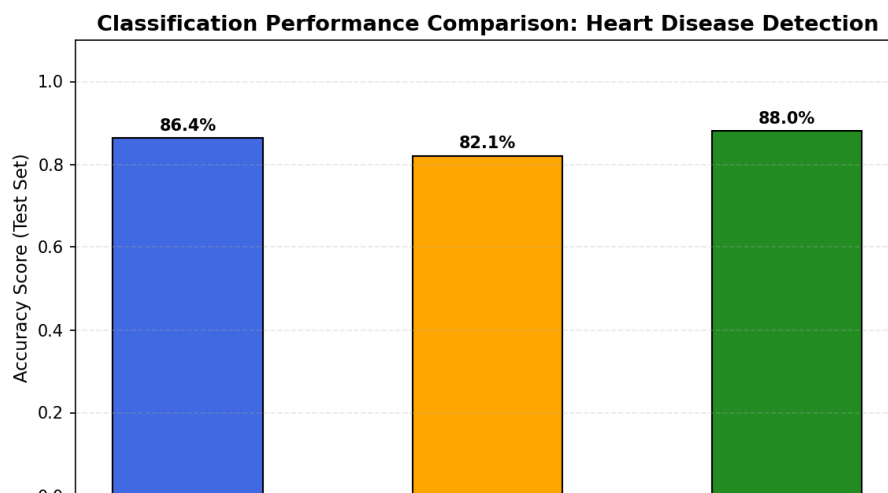


Fig. 3 Acuratete date inlocuite

În continuare pentru următoarele teste, voi folosi eliminarea de valori, nu doar pentru ca a oferit valori mai bune (care pot fi greșite datorită sample size-ului mai mic), dar pentru a menține integritatea datelor originale.

1.2.3. Alegerea și compararea modelelor

În curs au fost prezentate un număr de modele. Le vom compara folosind scriptul `classification_task_model_comparison.py`

După cum se observa, rezultatele sunt...

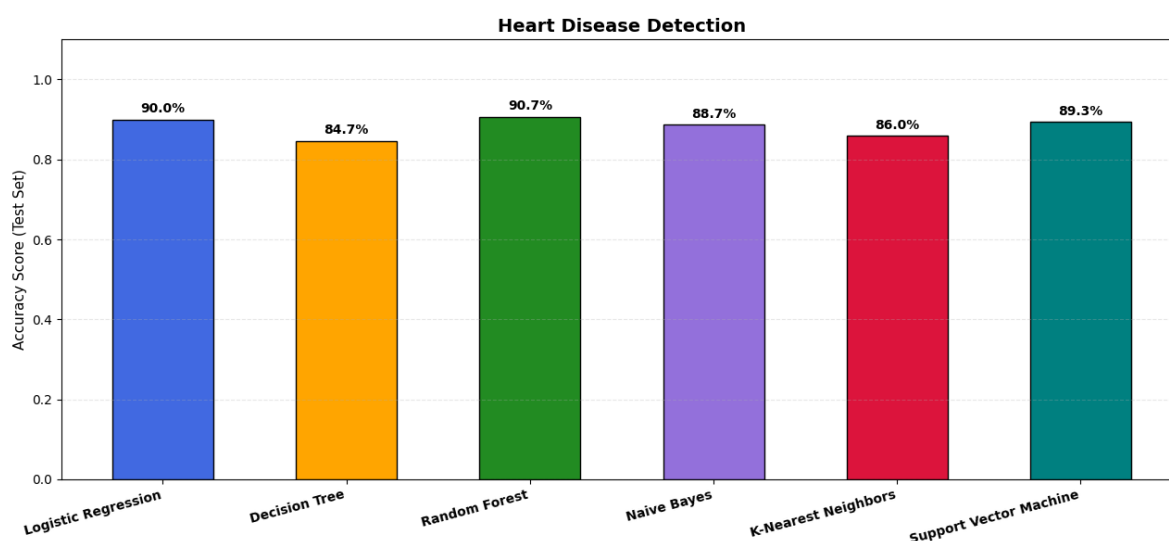


Fig. 4 Compararea acurateții diversilor algoritmi

În interesul diversității modului de funcționare al modelelor, voi alege:

- **Random Forest** - construiește o mulțime de arbori de decizie și combina răspunsurile lor prin vot majoritar pentru a obține o predicție finală cât mai exactă (arhitectura bazată pe arbori de decizie)
- **K-Nearest Neighbors (KNN)** - analizează elementele cele mai asemănătoare din jurul unui caz nou și îi atribuie clasa care predomină printre acești vecini apropiați (arhitectura de pattern recognition)
- **Support Vector Machine (SVM)** - găsește o linie sau o graniță geometrică optimă, numită hiperplan, separând clasele de date la o distanță maximă unele de altele (model geometric)

1.2.4. Fine tuning

Ajustarea hiperparametrilor este esențială deoarece previne underfitting sau overfitting, asigurând ca modelele pot generaliza corect pe date complet noi. Găsirea echilibrului reprezintă adesea diferența dintre un model mediocru și unul cu o acuratețe excelentă.

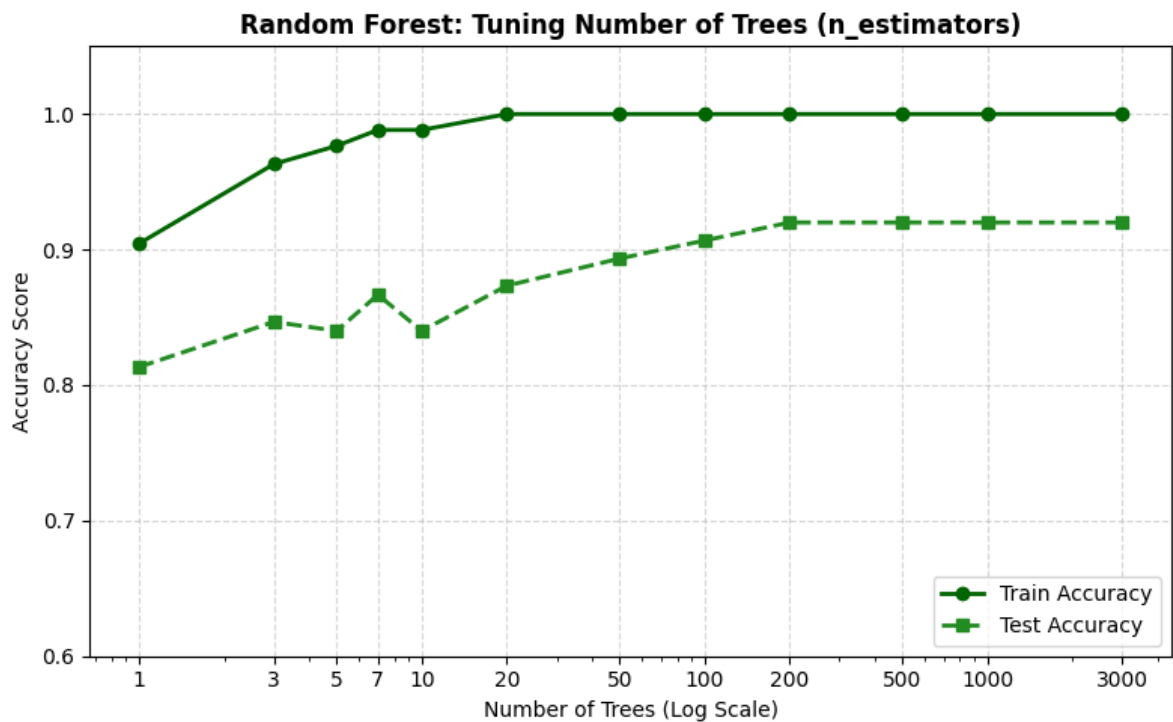


Fig. 5 Tunare parametri Random Forest

- Random Forest - Chiar dacă arborii individuali din cadrul pădurii memorizează complet setul de antrenament (acuratețea de antrenare devine 100%), votul colectiv al ansamblului acționează ca un amortizor de varianță. Adăugarea mai multor arbori nu cauzează overfitting, aceasta pur și simplu netezește erorile individuale și stabilizează limita finală de predicție.

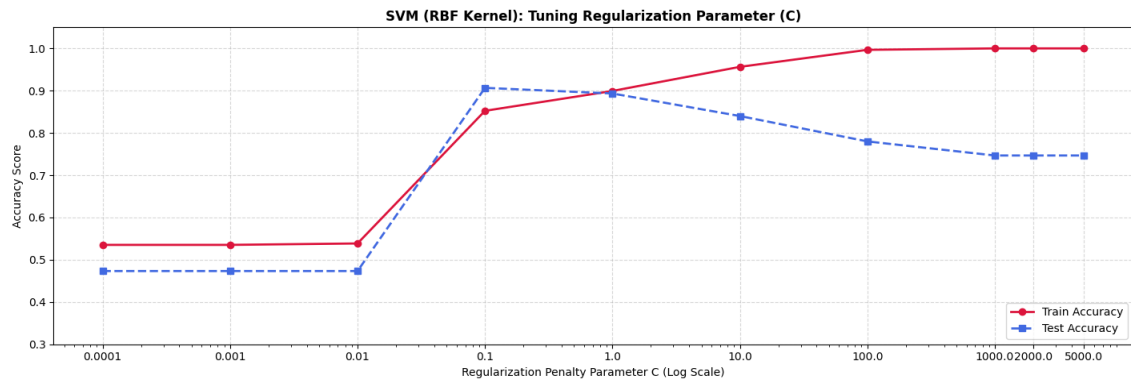


Fig. 6 Tunare parametri SVM

SVM - Pe măsură ce variabila de regularizare C crește, algoritmul SVM este forțat să ia o decizie complicată pentru a se asigura că absolut niciun pacient din setul de antrenament nu este clasificat greșit. Acest lucru determină modelul să memoreze anomalii individuale de antrenare, creând o limitare care eșuează complet pe datele de test nevăzute.

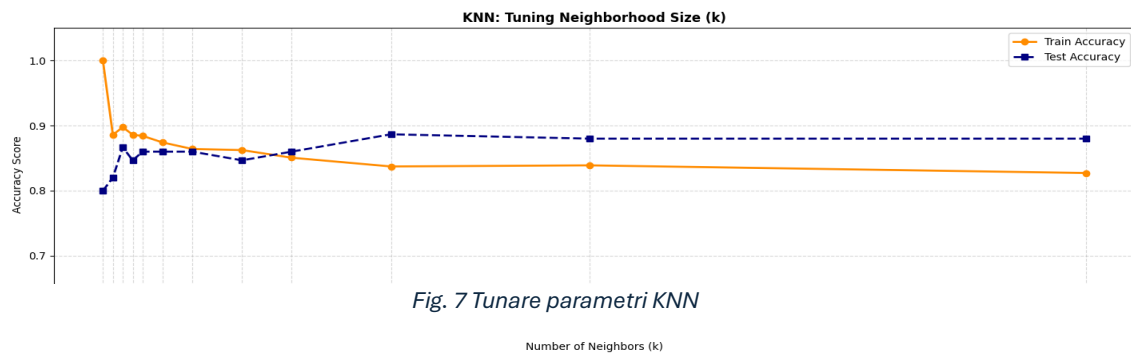


Fig. 7 Tunare parametri KNN

- KNN - La valori mici ale lui k, numărul de datapoint-uri „vecine” este foarte limitat. Creșterea lui k forțează modelul să ia un vot democratic mai larg al vecinătății locale a pacienților. Acest lucru filtrează anomalii izolate și creează o limită de decizie mai netedă și mai generalizată. Un număr mare al lui k însă, începe să generalizeze tot setul de pacienți întrucât „vecinii” pacientului vor fi mereu aceiași indiferent de specificațiile cazului

In final am ales valorile:

- $n_estimators = 200$ pentru Random Forest – deoarece oferă performanța maximă, iar valorile următoare formează un platou
- $k = 29$ pentru KNN , deși 30 este valoarea testată, un număr impar asigură existența unui vot decisiv în cazul unei egalități
- $C = 0.1$, pentru SVM este valoarea care oferă punctul de echilibru stabil între over și underfitting

1.2.5. Rezultate

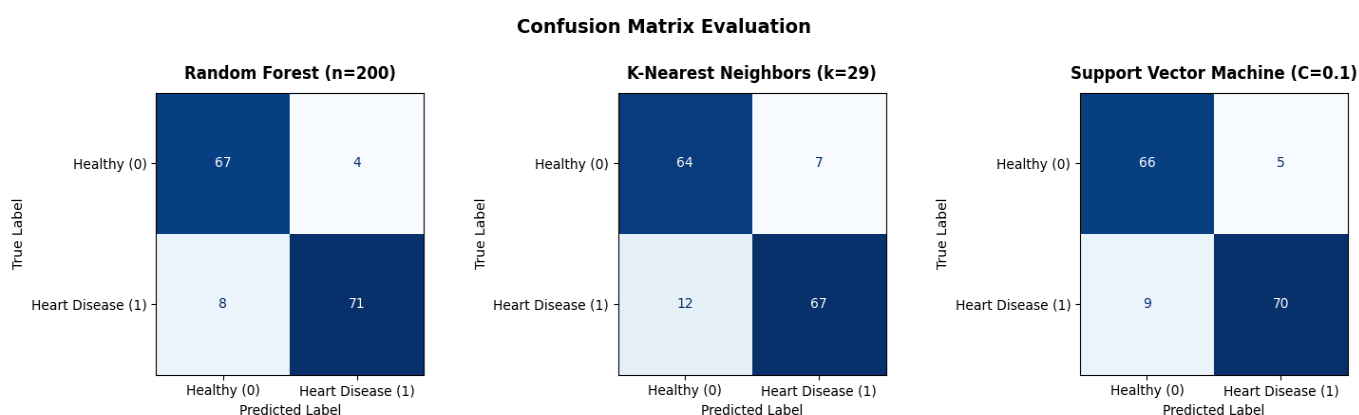


Fig. 8 Matricile de confuzie

După cum se poate vedea în figura matricelor de confuzie, Random Forest câștigă la nivelul de acuratețe cu un rating de 92%, SVM se afla pe locul 2 cu o acuratețe de 90.67% iar KNN este ultimul cu o acuratețe de 87.33%

Toate modelele au o rată aproape dublă de fals negative față de cele fals pozitive. Pe de o parte, ar putea fi calitatea/cantitatea de date. Persoanele cu probleme de inimă pot avea simptome vast diferite de la caz la caz, iar numărul limitat de date poate cauza probleme în training. Pe de altă parte, ar putea fi natura bolii: pe hârtie, unele persoane cu afecțiuni ale inimii pot arăta relativ sănătos, pe când o persoană ce este perfect sănătoasă rareori prezintă semne de boală.

1.2.6. Compararea performantei

Deși a câștigat, Random Forest a folosit până la 6000% mai multă memorie pe disc și 1700% mai mult timp de execuție. Până și procesul de predicție a fost de 4 ori mai lent decât modelul SVM care a luat locul 2. Și cu toate acestea, algoritmul a rezolvat corect doar un singur fals negativ și un singur fals pozitiv mai mult decât modelul care a luat locul 2.

```
=====
      RUNNING OPERATIONAL EFFICIENCY BENCHMARK
=====
Metric                Random Forest (n=200)    SVM (C=0.1)
-----
Model File Size        3188.97 KB                53.72 KB
Training Time          262.55 ms                14.96 ms
Inference Time (1k runs) 20.6091 sec             5.1309 sec
=====
```

Fig. 10 Benchmark modele

```
Retrieving the historical samples that voted on this patient...
```

Age	Sex	Cholesterol	MaxHR	HeartDisease	Distance_Score
29	1	204	202	0	1.413173
36	1	166	180	0	1.562568
34	0	161	190	0	1.640639
35	1	192	174	0	1.737361
41	1	157	182	0	1.855995
37	0	173	184	0	2.098370
41	0	184	180	0	2.195005
32	1	225	184	0	2.250527
44	1	219	188	0	2.258236
38	1	175	173	0	2.263437
34	0	210	192	0	2.276499
34	1	214	168	0	2.282515
36	1	209	178	0	2.350523
36	1	160	172	0	2.366657
35	1	161	153	0	2.382249
41	1	172	158	1	2.439773
34	1	156	180	1	2.440792
29	1	243	160	0	2.443536
37	1	223	168	0	2.478155
32	0	198	165	0	2.500449
35	0	183	182	0	2.506146
42	1	198	155	0	2.512624
41	0	204	172	0	2.535839
29	1	263	170	0	2.590723
42	1	196	150	0	2.603425
44	1	220	170	0	2.617654
39	1	204	145	0	2.621315
43	0	201	165	0	2.631927
43	0	215	175	0	2.654535

```
=====
```

Fig. 9 Transparența deciziilor KNN

1.2.7. Concluzii

- dacă singura țintă este acuratețea, atunci Random Forest este algoritmul necesar
- dacă resursele utilizate contează atunci SVM este o alternativă ideală
- deși KNN a venit pe ultimul loc, un avantaj al acestuia, mai ales în industria medicală este transparența alegerilor acestuia. Se pot afișa candidații aleși în întocmirea deciziei pentru interesul personalului medical.

Capitolul 2. Regresie

2.1. Definiție

Regresia în învățarea automată este o altă metodă fundamentală de învățare supervizată, însă, spre deosebire de clasificare, regresia este utilizată pentru a prezice o valoare numerică continuă. Modelul analizează relațiile dintre caracteristicile de intrare și o variabilă țintă numerică pentru a aproxima o funcție capabilă să estimeze valori exacte în viitor.

2.1.1. Proces

Procesul de regresie constă în următorii pași:

- colectarea și prelucrarea datelor: se identifică caracteristicile predictive (input) și variabila numerică continuă care trebuie estimată (Target).
- antrenarea modelului: algoritmul ajustează coeficienții sau structura internă pe setul de antrenament pentru a minimiza distanța dintre valorile prezise și valorile reale istorice.
- testarea și validarea: modelul estimează valori pentru setul de date de test, iar performanța sa este evaluată.

2.1.2. Algoritmi

Cei mai populari algoritmi de regresie sunt:

- regresia liniară (Linear Regression)
- arborii de decizie pentru regresie (Regression Trees)
- random forest regressor
- support vector regression (SVR)
- k-nearest neighbours regressor (KNN)

2.1.3. Metrici de evaluare

Evaluarea algoritmilor de regresie se face prin metrici bazate pe erori, precum:

- MAE - Mean Absolute Error: media diferențelor absolute dintre valorile reale și cele prezise
- MSE - Mean Squared Error: media pătratelor erorilor - penalizează mult mai dur greșelile de tip outlier
- R2 Score: indica proporția din varianța totală a variabilei țintă care poate fi explicată de model.

2.2. Aplicație

Pentru secțiunea de regresie a proiectului am ales setul de date `students.csv`, ce conține un număr de elevi împreună cu diverși factori socio-economici și rezultatele lor în diferite teste.

Sursa: <https://www.kaggle.com/datasets/spscientist/students-performance-in-exams>

2.2.1. Atribute

Atributele sunt:

- **gender**: Sexul studentului
- **race/ethnicity**: Grupul etnic sau rasa din care face parte elevul. Datele sunt anonimizate sub forma de categorii
- **parental level of education**: Cel mai înalt nivel de școală absolvit de părinți
- **lunch**: Tipul de pranz pe care elevul îl mănâncă la școală
- **test preparation course**: Dacă elevul a urmat un curs special de pregătire înainte de examene
- **math score**: Nota obținută de elev la testul de matematică
- **reading score**: Nota obținută la testul de lectură
- **writing score**: Nota obținută la testul de scris

2.2.2. Prelucrarea setului de date

1. Curățarea

Am verificat dacă există valori lipsă (NaN / null) sau rânduri duplicate.

Spre deosebire de setul de date medical anterior (`heart.csv`), unde valorile de 0 la colesterol sau presiune erau imposibile fizic, în acest set o notă de 0 la matematică, citire sau scris este teoretic posibilă (un examen picat complet sau un elev absent). Prin urmare, valorile de 0 nu au fost eliminate.

2. Transformarea variabilelor

Algoritmii de regresie lucrează exclusiv cu numere, așa ca am transformat toate coloanele de tip text in același mod descris la clasificare.

3. Definirea caracteristicilor si țintelor

Am stabilit variabila pe care dorim sa o estimam. In acest caz am dorit sa putem prezice rezultatul fiecărui test individual (folosind rezultatele celorlalte teste) sau sa estimam media generala a testelor, folosind doar factorii socio-economici cunoscuți despre un student (fără a cunoaște alta performanta academica).

4. Împărțirea setului de date – in mod similar cum am făcut pentru clasificare

5. Scalarea Datelor – in mod similar cum am făcut pentru clasificare

2.2.3. Modele alese

- **Multiple Linear Regression** prezice o valoare continua prin găsirea unei relații liniare directe intre variabilele de intrare, atribuind fiecărui factor o anumita pondere.
- **Random Forest Regressor** estimează rezultatul final prin calcularea mediei predicțiilor individuale oferite de o mulțime de arbori de decizie independenți.

2.2.4. Fine tuning

○ Random Forest Regression

- *n_estimators* - Determina numărul de arbori independenți care vor compune pădurea. Cu cat testam valori mai mari, cu atât media predicțiilor devine mai stabila, reducând varianța globala a modelului fără riscul de a genera overfitting, însă cu costul unui timp de calcul mai ridicat.
- *max_depth* - Controlează complexitatea si nivelul de ramificare al fiecarui arbore in parte. Valorile mici limitează modelul la decizii mult prea simple (underfitting), in timp ce valorile mari permit arborilor sa memoreze particularitățile sau anomalile izolate din datele de antrenament (overfitting).
- *min_samples_split* - Reprezintă numărul minim de date care trebuie sa se afle intr-un nod pentru ca acesta sa poată fi divizat mai departe. Fixarea acestei valori la 10 oprește intenționat algoritmul din a crea ramificații microscopice pentru doar 2 sau 3 rânduri care conțin valori atipice, acționând ca un filtru direct împotriva zgomotului din date.

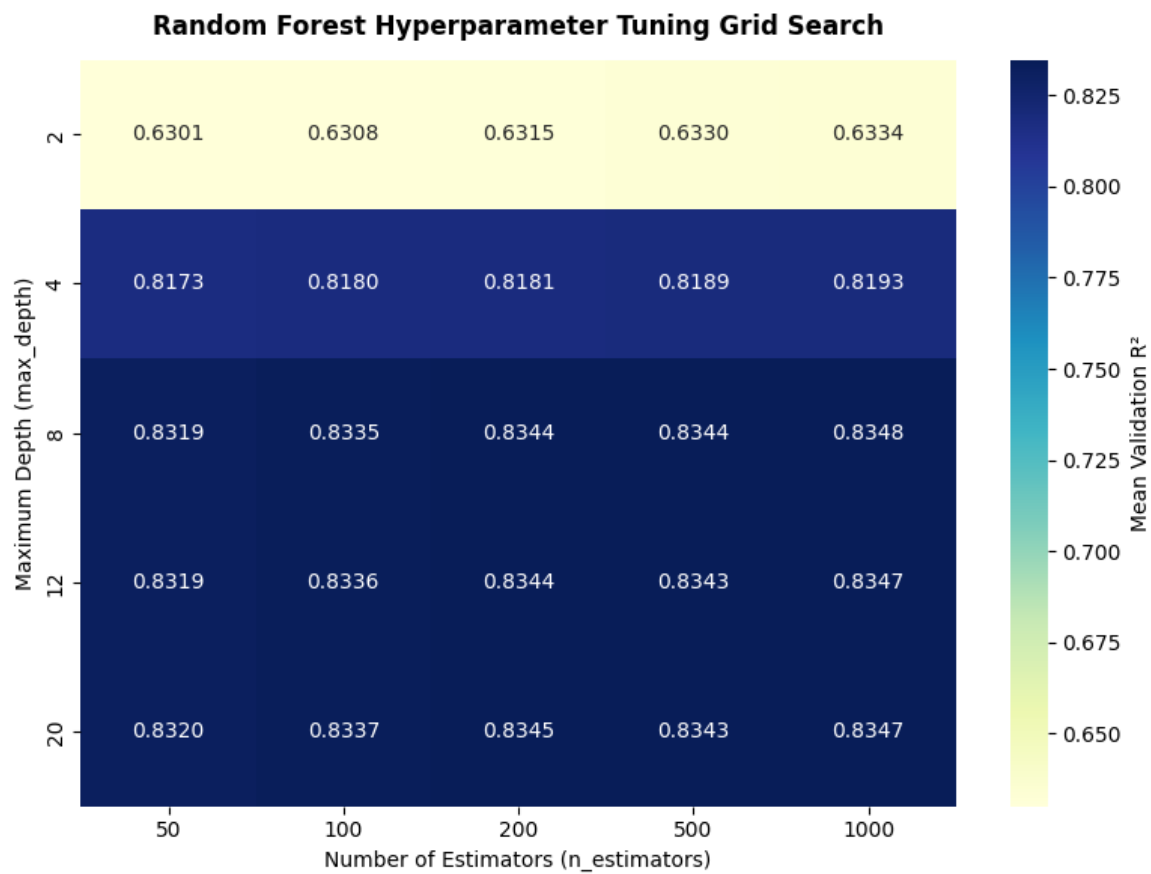


Fig. 11 Tunare parametri Random Forest

Considerând acestea, am ales valorile:

- `n_estimators=200`
- `max_depth=12`
- `min_samples_split=10`

2.2.5. Rezultate

În primul caz, am încercat să determin nota unui elev bazat pe toți ceilalți factori existenți (incluzând celelalte 2 note)

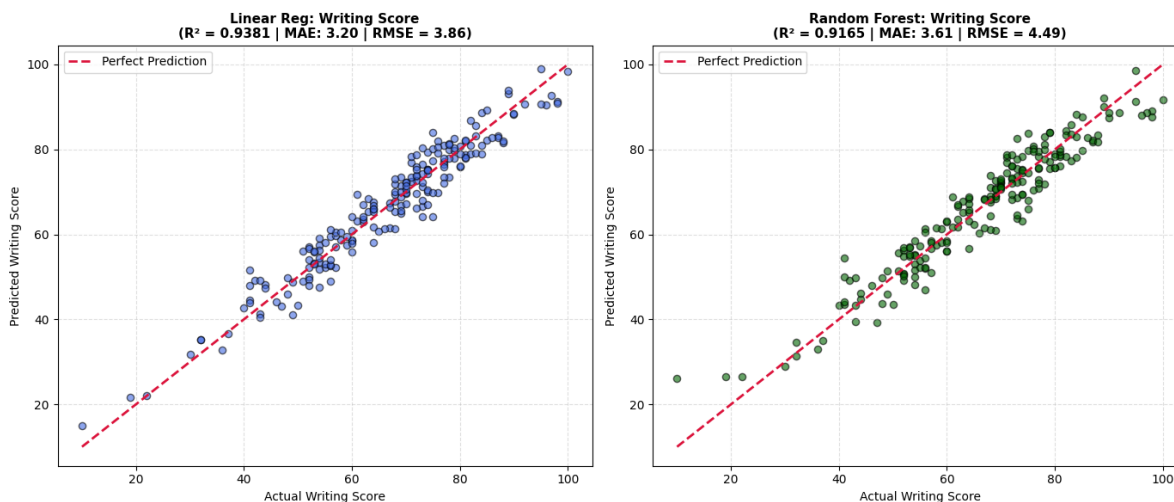


Fig. 12 Rezultate test scriere

- Testul de scriere este cel mai predictibil $R^2 = 0.9381$, MAE = 3.20

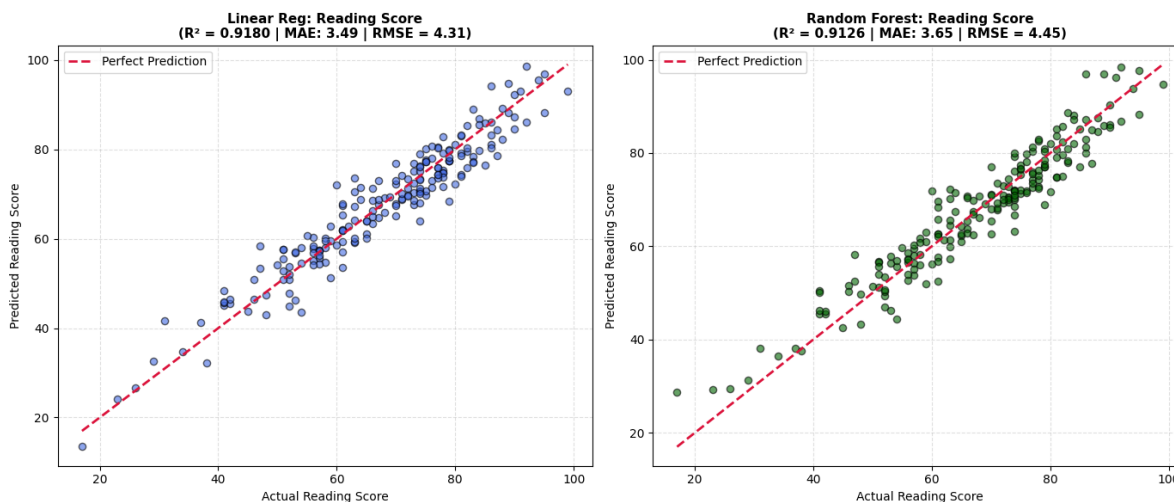


Fig. 13 Rezultate test lectura

- Testul de lectura este similar $R^2 = 0.9180$, MAE = 3.49

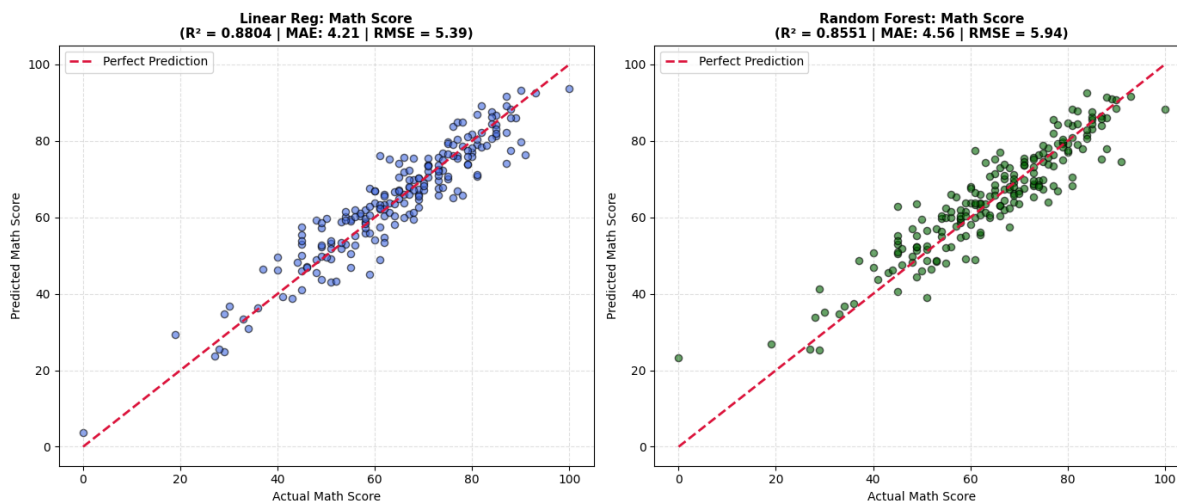


Fig. 14 Rezultate test matematica

- Testul de matematica este puțin mai jos decât ambele dar cu un scor încă suficient de bun $R^2 = 0.8804$, MAE = 4.21

In al doilea caz, am încercat sa determinam media notelor unui elev, bazat doar pe factorii socio-economici (excluzând performanta academica anterioara)

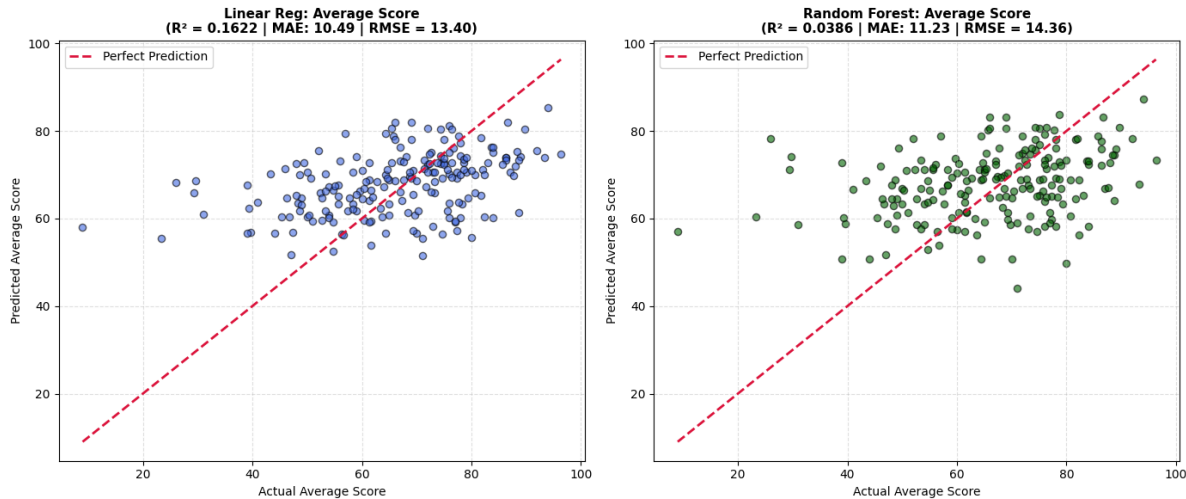


Fig. 15 Rezultate medie generala

- Modelul a eșuat sa producă rezultate satisfăcătoare, rezultând scorul R^2 de 0.1622 in cazul ideal si 0.0386 folosind Random Forest.

2.2.6. Performanta algoritmilor

După cum știm de la clasificare, Random Forest folosește exponențial mai multe resurse, însă spre deosebire de cealaltă metoda de învățare supervizată, algoritmul nu a mai oferit rezultatele cele mai bune. Unul din motive ar putea fi relațiile și simplitatea caracteristicilor input, pentru care algoritmul Random Forest complica deciziile mai mult decât necesar.

2.2.7. Concluzii

- Matematica, deși nu cu mult, este testul ale cărui rezultate sunt cele mai greu de prezis. Este posibil ca abilitățile necesare acolo (logica abstractă, lucrul cu numere) să nu se încadreze în curriculumul celorlalte 2 materii, pe când scrierea și lectura au aceeași bază literară și pot co-depinde pentru a crește încrederea predicției modelului. În plus, notele la matematica au valori volatile.
- Performanțele academice co-depind, dacă se cunosc notele de la lectura și matematica al unui student, modelul poate prezice nivelul de scriere cu o margine de eroare de 3.2 puncte.
- Factorii socio-economici nu dictează notele unui student. Există prea multă variație individuală, care nu este inclusă în setul de date (timpul de studiu, efortul depus, etc).

Capitolul 3. Clusterizare

3.1. Definiție

Clusterizarea în învățarea automată este o metodă fundamentală de învățare nesupervizată prin care un algoritm analizează un set de date fără etichete predefinite pentru a grupa obiectele în funcție de similaritatea dintre ele. Spre deosebire de clasificare și regresie, unde modelul are o țintă clară de prezis, clusterizarea descoperă structuri și tipare ascunse de una singură, asigurându-se că elementele din același cluster sunt cât mai asemănătoare, iar cele din grupuri diferite sunt cât mai distincte.

3.1.1. Proces

Procesul de clusterizare constă în următorii pași:

- colectarea și prelucrarea datelor: se folosesc caracteristici de input fără o variabilă țintă, axându-se pe atributele care pot defini asemănarea dintre instanțe

- antrenarea modelului: algoritmul explorează datele si calculează distanțele matematice dintre puncte pentru a defini si a forma grupurile optime
- analiza si validarea: deoarece nu exista etichete reale cu care sa se compare rezultatul, performanta si relevanta clusterelor sunt evaluate prin metrici de proximitate sau prin analiza directa a caracteristicilor fiecărui grup forma
-

3.1.2. Algoritmi

Cei mai populari algoritmi de clusterizare sunt:

- K-means clustering
- DBScan
- Hierarchical clustering
- Principal Component Analysis
- t-Distributed Stochastic Neighbor Embedding
- Hidden Markov Models

3.2. Aplicație

Pentru secțiunea de clusterizare a proiectului am ales seturile de date countries.csv si hrd.csv, ce conțin informații despre toate tarile recunoscute, respectiv informații despre 240 de stele.

Surse: <https://www.kaggle.com/datasets/rohan0301/unsupervised-learning-on-country-data/data>

<https://www.kaggle.com/code/salmanhiro/hertzsprung-russell-diagram/input>

3.2.1. Atribute

countries.csv:

- **country:** Numele țării
- **child_mort:** Numărul de decese ale copiilor sub 5 ani la 1000 de nașteri vii
- **exports:** Valoarea exporturilor de bunuri și servicii, exprimată ca procent din PIB
- **health:** Cheltuielile totale pentru sănătate, exprimate ca procent din PIB
- **imports:** Valoarea importurilor de bunuri și servicii, exprimată ca procent din PIB
- **income:** Venitul net per persoană, ajustat în funcție de puterea de cumpărare
- **inflation:** Rata anuală a inflației, care măsoară creșterii prețurilor de la an la an
- **life_expec:** Numărul mediu de ani pe care îi va trăi un cetățean

- **total_fer**: Numărul mediu de copii născuți de o femeie de-a lungul vieții
- **gdpp**: PIB-ul pe cap de locuitor

hrd.csv:

- **Temperature (K)**: Temperatura la suprafața stelei, măsurată în Kelvin
- **Luminosity(L/Lo)**: Luminozitatea stelei calculată în raport cu luminozitatea Soarelui
- **Radius(R/Ro)**: Raza stelei exprimată ca multiplicator al razei Soarelui
- **Absolute magnitude(Mv)**: Magnitudinea absolută, care măsoară strălucirea reală și intrinsecă a stelei
- **Star type**: Tipul stelei reprezentat printr-un cod numeric
- **Star color**: Culoarea vizuală a stelei, determinată direct de temperatura
- **Spectral Class**: Clasa spectrală a stelei conform clasificării standard din astronomie

3.2.2. Prelucrarea setului de date

Prepararea datelor a fost făcută exact conform prelucrărilor făcute pentru învățarea supervizată, fără schimbări specifice.

3.2.3. Modele alese

Pentru îndeplinirea clusterizării, am folosit următoarele modele:

- **K-Means** - Grupează stelele în k-clustere distincte prin plasarea iterativă a unor puncte centrale numite centroizi și alocarea fiecărei valori la cel mai apropiat centru, urmărind minimizarea distanțelor din interiorul fiecărei grupări.
- **DBSCAN** - Identifică grupările de stele pe baza densității locale a punctelor din spațiul caracteristicilor, având avantajul de a descoperi clustere de forme neregulate și de a izola automat datele atipice sub formă de valori zgomot.

3.2.4. Fine tuning

- K-means:
 - *n_clusters* – reprezintă numărul de clustere dorite
- DBSCAN:
 - *eps* – reprezintă distanța maximă la care algoritmul va căuta vecini în jurul unui punct

- *min_samples* – reprezintă numărul minim de puncte în interiorul cercului definit de *eps*, pentru ca acea zonă să fie considerată un cluster valid

Pentru tipul de învățare nesupervizată am ales să testez manual diferiți parametri, întrucât nu sunt sigur că există o valoare ‘ideală’ pentru toate cazurile.

3.2.5. Rezultate

Cazul 1. Dataset-ul *countries.csv*, optimizat pentru clusterizare bazată pe importuri și exporturi

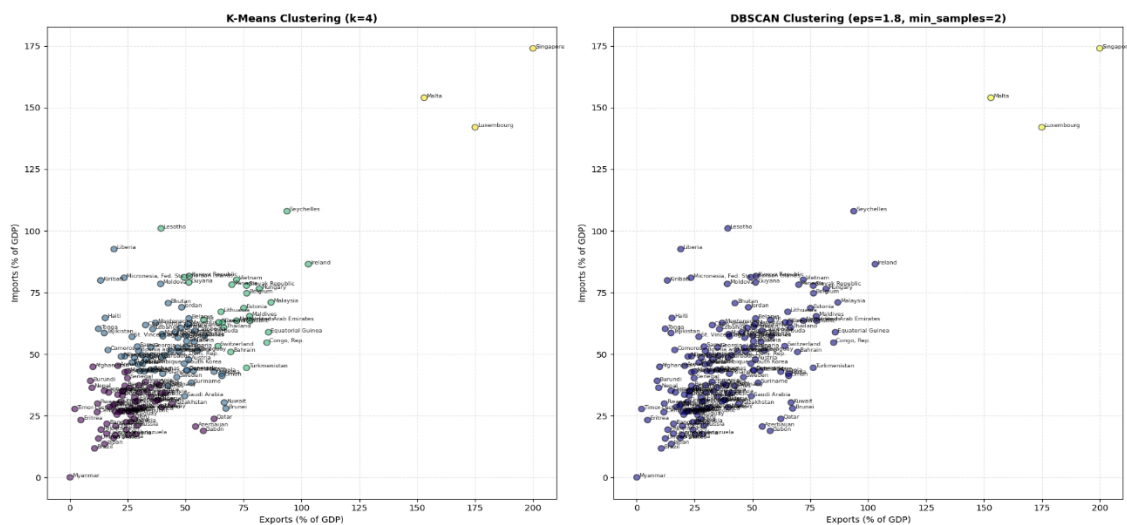


Fig. 16 Rezultate - importuri, exporturi

Cazul 2. Dataset-ul *countries.csv*, optimizat pentru clusterizare bazată pe calitatea de viață

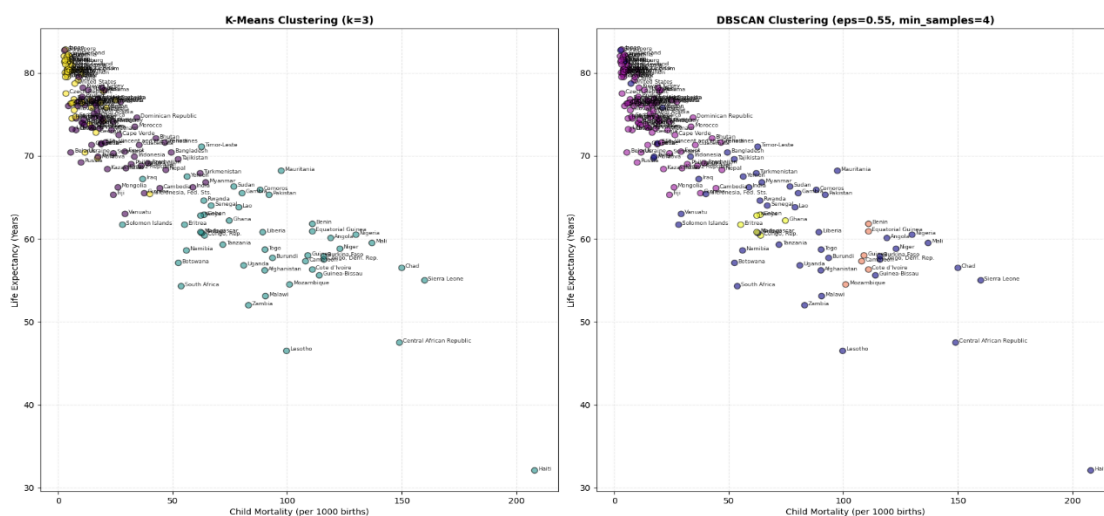


Fig. 17 Rezultate - calitatea de viață

Cazul 3. Dataset-ul countries.csv, optimizat pentru clusterizare generala, bazata pe toate caracteristicile oferite

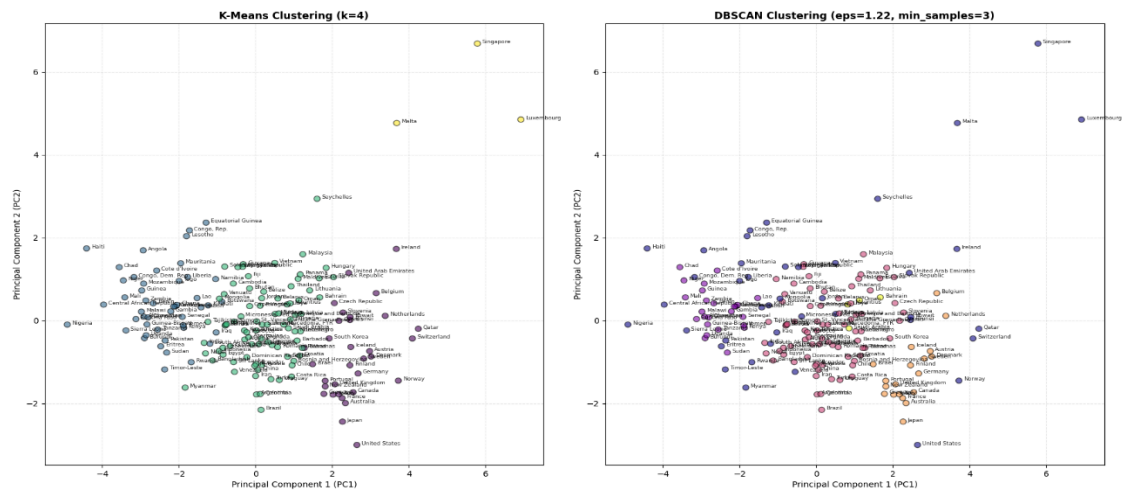


Fig. 18 Rezultate - categorii multiple

Cazul 4. Dataset-ul hrd.csv, optimizat pentru crearea unei diagrame Hertzsprung-Russell

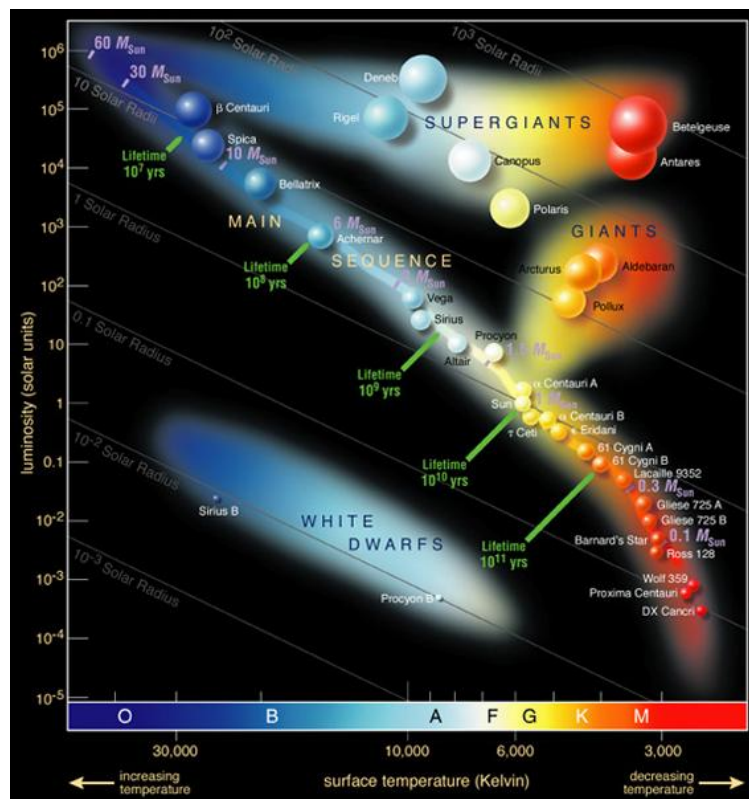


Fig. 19 Diagrama Hertzsprung-Russell [1]

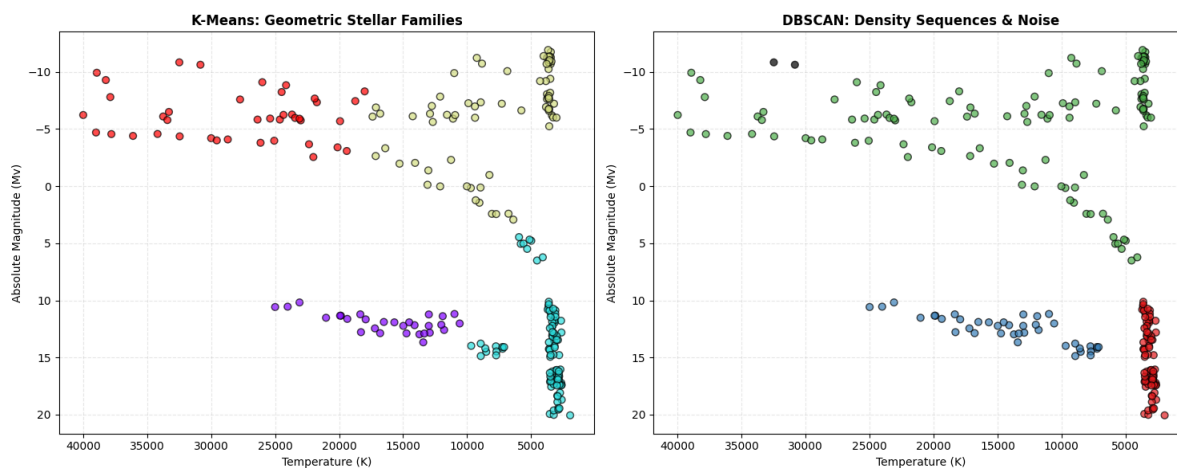


Fig. 20 Rezultate - stele

3.2.6. Concluzii

Cele două modele oferă perspective complementare asupra structurii datelor:

- K-Means s-a dovedit ideal pentru o segmentare geometrică clară și echilibrată în profiluri economice distincte (țări dezvoltate, în curs de dezvoltare și subdezvoltate), având însă dezavantajul de a forța chiar și valorile extreme să aparțină unui grup.
- În schimb, DBSCAN a excelat prin flexibilitatea sa bazată pe densitate, reușind să descopere grupări de forme neregulate fără a impune un număr fix de clustere și izolând automat statele cu situații atipice sub formă de zgomot (*outliers*).

Bibliografie

Curs Învățare Automată

[1] <https://www.cosmos.esa.int/web/cesar/the-hertzsprung-russell-diagram>